

M04 - Python Foundations

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

Rule

You are not expected to 'figure out programming from checkpoints'. Read, type, observe, explain, then practise.

Contents

- DE04-01** - Python setup and first program
- DE04-02** - Variables, data types and operators
- DE04-03** - Strings and formatting
- DE04-04** - Conditions and boolean logic
- DE04-05** - Lists, tuples, sets and dictionaries
- DE04-06** - Loops and iteration
- DE04-07** - Functions and scope
- DE04-08** - Modules, imports and project structure
- DE04-09** - Exceptions and debugging basics
- DE04-10** - Files, paths, CSV and JSON I/O
- DE04-11** - pandas foundations: load, inspect, filter and aggregate
- DE04-12** - Practical Big-O intuition
- DE04-13** - Coding drills: lists and strings
- DE04-14** - Coding drills: dictionaries and sets
- DE04-15** - Stack, queue, search and sort basics
- DE04-16** - Integrated Python checkpoint and reproducible script habits

DE04-01 - Python setup and first program

What you will be able to do

Understand what actually runs Python code and successfully run your first saved .py file.

Why this matters

Beginners often confuse VS Code, the Python extension, and Python itself. If you know which piece does what, setup and errors become much less mysterious.

Picture it this way

VS Code is the notebook you write in. The Python extension gives the notebook helpful Python features. The Python interpreter is the person who actually reads and executes the instructions.

Start from zero

- **Python code** is plain text. A file ending in .py is commonly called a Python script.
- **VS Code** is an editor. It does not contain the Python language runtime by itself.
- The **Python interpreter** is the installed program that executes Python instructions.
- The Microsoft **Python extension** helps VS Code find/run/debug Python but is separate from the interpreter.
- A **syntax error** means Python could not understand the written structure before normal execution continued.
- At this lesson, install Python only if it is not already installed. Use the official Python/VS Code instructions linked in [START_HERE](#).

Teacher demonstration

```
print("Hello data engineering")  
print(2 + 3)
```

Read the code like English

- `print(...)` asks Python to display a value in the terminal.
- Text inside quotes is a string value.
- `2 + 3` is calculated first; `print` then displays 5.
- Save the file as `first_program.py`, then run it from a known folder.

Expected output / validation

command/result	expected
<code>python --version</code> or <code>py --version</code>	a Python 3 version
<code>first_program.py</code> line 1	Hello data engineering
line 2	5

Common beginner traps

- Installing only the VS Code extension and believing Python itself is installed.
- Running a different file than the one you edited.
- Closing an error before saving/reading the traceback.

Now you do a similar task

Create `hello_me.py`. Print your name, the words 'Python started', and the result of $10 * 4$. Then intentionally remove one closing parenthesis, run it, save the error text, restore the parenthesis, and run successfully.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

Which component actually executes your `.py` file: VS Code, the Python extension, or the Python interpreter?

Answer in your own words before continuing.

DE04-02 - Variables, data types and operators

What you will be able to do

Store values in variables, recognize basic data types, and combine values with arithmetic/comparison operators.

Why this matters

Every later Python program stores values such as quantity, file path, status, row count, or price. Type confusion is a common source of bugs.

Picture it this way

A variable is a labeled box. The label qty points to a value. Python also remembers the kind of value in the box.

Start from zero

- A **variable** is a name bound to a value: `qty = 2`.
- An **int** is a whole number; **float** usually represents a decimal number; **str** is text; **bool** is True/False.
- `=` assigns a value. `==` compares two values for equality. They are not the same operation.
- Arithmetic operators include `+`, `-`, `*`, `/`.
- Comparison operators such as `>=` return a boolean value.
- **type(value)** helps you inspect what Python thinks the type is.

Teacher demonstration

```
product = "Keyboard"
qty = 2
unit_price = 850
sales = qty * unit_price
is_large = sales >= 1500
print(sales)
print(type(sales))
print(is_large)
```

Read the code like English

- `qty` and `unit_price` are integers.
- `qty * unit_price` produces 1700.
- `sales >= 1500` evaluates to True.
- `type(sales)` confirms the value is numeric instead of text.

Expected output / validation

expression	result
<code>sales</code>	1700
<code>type(sales)</code>	int
<code>is_large</code>	True

Common beginner traps

- Writing `qty = '2'` and expecting every numeric operation to behave like integer 2.
- Using `=` where you meant `==` inside a comparison.
- Converting values repeatedly instead of understanding why the source type is wrong.

Now you do a similar task

Set `qty=3` and `unit_price=450`. Calculate `sales`, tax at 18%, `final_amount`, and a boolean named `above_1000`. Print every value and its type.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

Why can `'3' + '4'` produce a different result from `3 + 4`?

Answer in your own words before continuing.

DE04-03 - Strings and formatting

What you will be able to do

Clean simple text and create readable messages with string methods and f-strings.

Why this matters

Data arrives as text constantly: region labels, IDs, file names, log messages, status fields and user input.

Picture it this way

A string is a piece of text you can inspect and produce new cleaned versions from. The original string value is not edited in place.

Start from zero

- **strip()** returns a new string without leading/trailing whitespace.
- **lower()**, **upper()**, and **title()** return new case variants.
- Strings are **immutable**: methods return a new value rather than changing the existing object in place.
- An **f-string** lets you insert values into readable text: `f'Order {order_id}'`.
- Do not normalize identifiers blindly if case/spacing can carry business meaning.

Teacher demonstration

```
raw_region = "  north zone "  
clean_region = raw_region.strip().title()  
order_id = "01007"  
message = f"{order_id} belongs to {clean_region}"  
print(message)
```

Read the code like English

- `strip()` removes outer spaces.
- `title()` returns 'North Zone'.
- The `f` before the string allows `{order_id}` and `{clean_region}` to be replaced with their values.

Expected output / validation

value	result
<code>raw_region</code>	<code>north zone</code>
<code>clean_region</code>	<code>North Zone</code>
<code>message</code>	<code>O1007 belongs to North Zone</code>

Common beginner traps

- Calling `raw_region.strip()` without saving/using the returned value, then wondering why `raw_region` still has spaces.
- Using `title()` on codes/identifiers that should not be changed.
- Building unreadable messages with many `+` operators when an f-string is clearer.

Now you do a similar task

Start with customer=' ravi retail ' and region=' EAST '. Create clean values and print exactly: Customer: Ravi Retail | Region: East.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

If strings are immutable, what happens when you call strip()?

Answer in your own words before continuing.

DE04-04 - Conditions and boolean logic

What you will be able to do

Turn a business rule into clear if/elif/else branches and test boundary values.

Why this matters

Pipelines classify, validate and route records based on conditions. The dangerous bugs are often at exact thresholds.

Picture it this way

A condition is a gate. Python checks a yes/no question. if/elif/else tells the program which path to take.

Start from zero

- **if** checks the first condition.
- **elif** checks another condition only if earlier branches did not run.
- **else** catches everything remaining.
- **and** requires both conditions; **or** requires at least one; **not** flips a boolean.
- Branch order matters when conditions overlap.
- Test below, exactly at, and above a threshold.

Teacher demonstration

```
status = "Completed"
sales = 2000
if status == "Completed" and sales >= 2000:
    label = "priority"
elif status == "Completed":
    label = "normal completed"
else:
    label = "not completed"
print(label)
```

Read the code like English

- Both first conditions are True: status is Completed and sales is exactly 2000.
- Because >= includes the boundary, 2000 is priority.
- Once the first branch runs, later elif/else branches are skipped.

Expected output / validation

test	label
Completed, 1999	normal completed
Completed, 2000	priority
Pending, 3000	not completed

Common beginner traps

- Using > 2000 when the written rule says 2000 or more.
- Checking a broad condition before a more specific condition and making the specific branch unreachable.
- Writing one giant unreadable boolean expression.

Now you do a similar task

Classify an order: High if Completed and sales ≥ 2500 ; Medium if Completed and sales ≥ 1200 ; Small if Completed; Otherwise Not completed. Test 1199,1200,2499,2500 plus a Pending order.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

If the rule says '2500 or more', why is $\geq$ different from $>$?

Answer in your own words before continuing.

DE04-05 - Lists, tuples, sets and dictionaries

What you will be able to do

Choose list, tuple, set or dictionary because of the access pattern you need.

Why this matters

Collections are not interchangeable decorations. The right structure makes intent clearer and later code simpler.

Picture it this way

A list is an ordered shopping list. A set is a bag of unique membership cards. A dictionary is a lookup desk: give a key, get its value.

Start from zero

- **list**: ordered, mutable, duplicates allowed.
- **tuple**: ordered and immutable; useful for fixed groups/records by convention.
- **set**: unique hashable values; fast average membership checks.
- **dict**: maps unique keys to values.
- Choose based on what you need to do, not what syntax looks familiar.

Teacher demonstration

```
products = ["Keyboard", "Mouse", "Keyboard"]
unique_products = set(products)
price_by_product = {
    "Keyboard": 850,
    "Mouse": 450,
}
print(unique_products)
print(price_by_product["Mouse"])
```

Read the code like English

- The list preserves three entries including repeated Keyboard.
- `set(products)` keeps unique product values.
- The dictionary lets you retrieve Mouse price by key instead of scanning pairs manually.

Expected output / validation

operation	meaning
<code>len(products)</code>	3 entries
<code>len(unique_products)</code>	2 unique products
<code>price_by_product["Mouse"]</code>	450

Common beginner traps

- Expecting set order to behave like a list order.
- Using a dictionary when you really need multiple values for the same key without designing for that.
- Repeatedly scanning a list for membership when a set expresses the need better.

Now you do a similar task

Given IDs ['O1','O2','O2','O3','O1'], create a set of unique IDs and a dictionary mapping East/West/North to manager names. Retrieve the West manager.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

If you only care whether an ID has already been seen, why may a set be a better fit than a list?

Answer in your own words before continuing.

DE04-06 - Loops and iteration

What you will be able to do

Process repeated values with for/while loops and maintain explicit accumulator state.

Why this matters

Files and datasets contain repeated rows. Loops let you apply the same logic while tracking totals, counts or flags.

Picture it this way

A loop is walking through a stack of receipts one at a time. An accumulator is the running total written on your calculator.

Start from zero

- **for value in collection** visits each item.
- An **accumulator** such as total starts from a known value and is updated inside the loop.
- A counter is a special accumulator for how many items matched.
- **while** repeats while a condition remains True; the condition must eventually change to avoid an infinite loop.
- Do not mutate the same list unpredictably while iterating through it.

Teacher demonstration

```
sales_values = [1700, 1350, 1100, 850, 1800]
total = 0
large_count = 0
for value in sales_values:
    total += value
    if value >= 1500:
        large_count += 1
print(total)
print(large_count)
```

Read the code like English

- total starts at 0.
- Every value is added once.
- large_count increases only for values >=1500.
- Expected total is 6800 and large_count is 2.

Expected output / validation

output	result
total	6800
values >= 1500	2

Common beginner traps

- Resetting total=0 inside the loop.
- Forgetting to update a while-loop condition.
- Using a loop for a task a clear built-in handles better after you understand the loop mechanics.

Now you do a similar task

Loop through [3,8,2,9,10,5]. Compute total, count even numbers, and largest value without using `sum()` or `max()` for this exercise.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

What would happen if `total = 0` were placed inside the for loop?

Answer in your own words before continuing.

DE04-07 - Functions and scope

What you will be able to do

Create small reusable functions with explicit parameters and return values.

Why this matters

Functions let data engineering code separate responsibilities and become testable instead of one long script.

Picture it this way

A function is a small machine: you pass inputs into labeled slots, it performs one job, and return sends the output back.

Start from zero

- **def** defines a function.
- **parameters** are input names in the definition.
- **arguments** are actual values passed during a call.
- **return** gives a value back to the caller; print only displays something.
- Local variables normally exist inside that function call.
- Small functions with explicit inputs are easier to test than functions that secretly depend on global state.

Teacher demonstration

```
def net_sales(qty, unit_price, discount_pct=0):  
    gross = qty * unit_price  
    return gross * (1 - discount_pct)  
result = net_sales(2, 1200, 0.10)  
print(result)
```

Read the code like English

- discount_pct has a default of 0 if the caller omits it.
- gross is local to the function.
- return produces 2160.0 and lets the caller store it in result.

Expected output / validation

call	result
net_sales(2,1200,0.10)	2160.0
net_sales(2,1200)	2400

Common beginner traps

- Printing a result inside the function when later code needs to use the value.
- Relying on a global variable that was never passed as a parameter.
- Writing one function that reads files, cleans data, calculates metrics and writes output all at once.

Now you do a similar task

Write order_summary(order_id, qty, unit_price) that returns a dictionary with order_id, qty and sales. Call it with two different inputs and compare outputs.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

What is the difference between `print(value)` and `return value`?

Answer in your own words before continuing.

DE04-08 - Modules, imports and project structure

What you will be able to do

Split code into modules and understand imports without triggering unwanted work.

Why this matters

Real projects contain multiple files. Helpers should be reusable, while the main script controls when the work actually runs.

Picture it this way

A module is another toolbox file. Importing it should open the toolbox, not automatically start a machine that deletes or rewrites data.

Start from zero

- A **module** is commonly a .py file that Python can import.
- An **import** executes top-level code in that module once during loading.
- Therefore top-level destructive work in helper modules is risky.
- **if __name__ == '__main__':** protects code that should run only when the file is executed directly.
- Keep reusable functions in helpers; keep orchestration in an entry script.

Teacher demonstration

```
# helpers.py
def clean_region(value):
    return value.strip().title()
# main.py
from helpers import clean_region
def main():
    print(clean_region(" east "))
if __name__ == "__main__":
    main()
```

Read the code like English

- Importing helpers gives access to clean_region.
- main() describes executable flow.
- The __name__ guard runs main() only when main.py is executed directly.

Expected output / validation

action	expected
python main.py	prints East
import main from another module	does not automatically run main()

Common beginner traps

- Putting file deletion/write work at import time.
- Creating circular imports between modules.
- Naming your own file pandas.py/csv.py and shadowing a standard/installed module.

Now you do a similar task

Complete DATA/stage04_helpers.py and run DATA/M04_MODULE_DEMO_v1.0.py. Then import stage04_helpers in a Python shell and confirm it does not print or write anything by itself.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

Why is top-level destructive code dangerous inside a helper module that other files import?

Answer in your own words before continuing.

DE04-09 - Exceptions and debugging basics

What you will be able to do

Read tracebacks as evidence, fix one root cause at a time, and catch only errors you can handle meaningfully.

Why this matters

Engineering work includes broken inputs and broken code. Debugging skill is more valuable than pretending errors should never happen.

Picture it this way

A traceback is a route map showing where the failure traveled. Start at the final exception message, then inspect the relevant line and inputs.

Start from zero

- A traceback ends with an **exception type** and message such as `TypeError` or `ValueError`.
- Read the bottom line first, then inspect the referenced line in your code.
- **ValueError** often means the type is acceptable but the value cannot be converted/used as requested.
- **TypeError** often means an operation received incompatible types.
- Use `try/except` when you know how to handle a specific failure; do not catch `Exception` everywhere and hide bugs.
- Save the original error before changing code.

Teacher demonstration

```
raw_qty = "3"
try:
    qty = int(raw_qty)
    print(qty * 100)
except ValueError:
    print(f"Invalid quantity: {raw_qty!r}")
```

Read the code like English

- `int('3')` succeeds.
- If `raw_qty` were `'five'`, `int(...)` would raise `ValueError`.
- The `except` branch gives a useful message that includes the bad value.

Expected output / validation

input	result
'3'	300
'five'	Invalid quantity: 'five'

Common beginner traps

- Replacing the whole file from an answer key instead of understanding the first error.
- Using `except: pass` and silently losing bad data.
- Catching `ValueError` around code that can fail for many unrelated reasons.

Now you do a similar task

Run DATA/M04_DEBUG_ME_v1.0.py exactly as provided. Save the first traceback. Fix one issue only, rerun, then continue until the script handles the bad quantity and missing price intentionally.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

When a traceback appears, which part should you read first and why?

Answer in your own words before continuing.

DE04-10 - Files, paths, CSV and JSON I/O

What you will be able to do

Read and write files using explicit paths while protecting raw input data.

Why this matters

Data engineering is largely moving data between files/systems. Path mistakes and accidental overwrites are common beginner failures.

Picture it this way

A Path object is a precise address. Reading is opening the source. Writing should usually go to a separate destination so the original evidence remains intact.

Start from zero

- **pathlib.Path** represents file/folder paths in a readable cross-platform way.
- **Path(__file__).with_name(...)** builds a path next to the current script.
- The built-in **csv** and **json** modules handle common text data formats.
- Specify text encoding such as utf-8 when opening text files.
- Separate raw input from derived output; do not casually overwrite the only source copy.
- Create output folders deliberately before writing.

Teacher demonstration

```
from pathlib import Path
import csv
data_file = Path(__file__).with_name("M04_RETAIL_ORDERS_v1.0.csv")
with data_file.open("r", encoding="utf-8", newline="") as f:
    rows = list(csv.DictReader(f))
print(len(rows))
print(rows[0]["order_id"])
```

Read the code like English

- **Path(__file__)** refers to the script's own path.
- **with_name** changes only the file name, keeping the folder.
- **csv.DictReader** returns each row as a dictionary keyed by column name.
- The supplied CSV contains 25 order rows.

Expected output / validation

check	expected
row count	25
first order_id	O2001

Common beginner traps

- Using a relative path without knowing the current working directory.
- Writing output back over the raw input file.
- Forgetting encoding/newline choices and then debugging strange text/blank-line issues later.

Now you do a similar task

Read the retail CSV with `csv.DictReader`, count Completed rows, create an output folder next to your script, and write a small CSV containing only Completed order_id and region. Keep the raw source unchanged.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

Why is writing derived output to a separate folder safer than overwriting the input CSV?

Answer in your own words before continuing.

DE04-11 - pandas foundations: load, inspect, filter and aggregate

What you will be able to do

Use pandas to load, inspect, filter and aggregate tabular data while validating totals.

Why this matters

pandas is a common Python tool for small/medium in-memory data tasks and is useful preparation for later Data Engineering transformations.

Picture it this way

A pandas DataFrame is a programmable table. Before changing it, you first inspect its size, columns, sample rows and data types.

Start from zero

- **pd.read_csv** loads a CSV into a DataFrame.
- **shape** gives (rows, columns); **columns** lists column names; **dtypes** shows inferred types; **head()** samples rows.
- Boolean filtering selects rows matching a condition.
- **groupby(...)[measure].sum()** summarizes values by groups.
- Create calculated columns explicitly and reconcile grouped totals with an overall total.
- Do not treat pandas as magic. Always state what one row represents.

Teacher demonstration

```
import pandas as pd
from pathlib import Path
path = Path(__file__).with_name("M04_RETAIL_ORDERS_v1.0.csv")
df = pd.read_csv(path)
df["sales"] = df["qty"] * df["unit_price"]
completed = df[df["status"] == "Completed"]
regional = completed.groupby("region")["sales"].sum().sort_values(ascending=False)
print(df.shape)
print(regional)
print("reconciled:", regional.sum(), completed["sales"].sum())
```

Read the code like English

- The raw file has 25 rows and 8 columns.
- sales is calculated per order row.
- completed removes Pending/Cancelled rows before the groupby.
- regional.sum() must equal completed['sales'].sum() if no rows were lost/duplicated in the grouping.

Expected output / validation

region	Completed sales
West	8100
North	7310
South	7160
East	6700

Common beginner traps

- Grouping before filtering when the business asks for Completed only.
- Trusting a grouped output without reconciling total sales.
- Editing column names/data blindly before inspecting the DataFrame.

Now you do a similar task

Load the CSV. Report shape, columns, dtypes, number of missing sales_rep values, top 3 Completed orders by sales, and Completed sales by region. Reconcile regional sum to overall Completed sales.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

What two checks should you do immediately after read_csv before starting analysis?

Answer in your own words before continuing.

DE04-12 - Practical Big-O intuition

What you will be able to do

Build practical intuition for why some operations scale better than others without turning this into a math-heavy algorithms course.

Why this matters

Data engineers should notice when code repeatedly scans millions of items. You do not need competitive-programming tricks, but you should recognize avoidable repeated work.

Picture it this way

Looking for a name on an unsorted paper list may require scanning many lines. A well-designed lookup structure is like an index that lets you check membership much more directly.

Start from zero

- **Big-O** describes how work grows as input size grows; it is a growth model, not an exact stopwatch.
- A general list membership check may scan items: $O(n)$.
- Set/dictionary membership is typically $O(1)$ average case because of hashing.
- A loop inside another loop can become $O(n^2)$ in common cases.
- Choose the simplest structure that matches the access pattern; do not micro-optimize tiny code without evidence.

Teacher demonstration

```
seen = set()
for order_id in ["01", "02", "02", "03"]:
    if order_id in seen:
        print("duplicate:", order_id)
    else:
        seen.add(order_id)
```

Read the code like English

- seen is a set because the main operation is membership: 'have I seen this ID already?'
- Each order ID is processed once.
- The code expresses intent more directly than repeatedly scanning a separate list of previous IDs.

Expected output / validation

output	result
duplicate detected	O2 once

Common beginner traps

- Saying set lookup is literally constant time in every theoretical worst case.
- Optimizing before code is correct/readable.
- Using a set when you actually need duplicate counts or input order preserved in the output.

Now you do a similar task

Explain in 3-5 sentences why a set is useful for repeated ID lookups. Then write a list-scan version and a set-membership version for detecting whether 'O999' exists.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

If you repeatedly ask 'is this ID already present?', what access pattern are you optimizing?

Answer in your own words before continuing.

DE04-13 - Coding drills: lists and strings

What you will be able to do

Strengthen basic list/string manipulation through small interview-style drills with explicit tests.

Why this matters

You need enough coding fluency to manipulate simple values without searching every syntax detail, but the goal is reliable data work - not puzzle competition.

Picture it this way

These are scales for a musician: short repeated exercises that make later real work smoother.

Start from zero

- Break a drill into input -> transformation -> output.
- Write at least one normal case and one edge case before calling the function finished.
- Prefer readable built-ins when you understand them.
- Do not memorize a solution you cannot explain.

Teacher demonstration

```
def even_values(values):
    result = []
    for value in values:
        if value % 2 == 0:
            result.append(value)
    return result
print(even_values([3, 8, 2, 9, 10, 5]))
```

Read the code like English

- % gives the remainder; an even number has remainder 0 when divided by 2.
- result starts empty and collects only matching values.
- The function returns [8, 2, 10].

Expected output / validation

test	result
mixed list	[8, 2, 10]
empty list	[]

Common beginner traps

- Printing inside the function instead of returning a reusable result.
- Testing only the sample case.
- Writing a one-line trick you cannot explain.

Now you do a similar task

Complete list/string drills: count vowels in 'data engineering'; reverse word order in 'build reliable data pipelines'; return first repeated order ID or None.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

Why should every small function have at least one edge-case test?

Answer in your own words before continuing.

DE04-14 - Coding drills: dictionaries and sets

What you will be able to do

Use dictionaries for counts/mappings and sets for uniqueness/duplicate detection.

Why this matters

Counts by key and duplicate checks appear everywhere in validation: regions, statuses, IDs, file names and event types.

Picture it this way

A dictionary is a tally board: each region key points to its current count. A set remembers which unique IDs have appeared.

Start from zero

- Use `dict.get(key,0)` to read a current count with a default.
- Increment counts as you loop.
- A separate seen set can detect the moment an ID repeats.
- A duplicates set can keep each duplicate ID only once.
- Choose a dictionary when each key needs an associated value, not only membership.

Teacher demonstration

```
regions = ["East", "West", "East", "North", "West", "East"]
counts = {}
for region in regions:
    counts[region] = counts.get(region, 0) + 1
print(counts)
```

Read the code like English

- The first East uses default 0 then becomes 1.
- Later East occurrences read the existing count and increment it.
- Expected counts: East 3, West 2, North 1.

Expected output / validation

key	count
East	3
West	2
North	1

Common beginner traps

- Using a set when you need counts.
- Resetting `counts[region]=1` on every occurrence.
- Returning duplicates many times instead of once when the task asks for unique duplicate IDs.

Now you do a similar task

For `ids=['O1','O2','O2','O3','O1','O4']`, return duplicate IDs only once and build a dictionary of how many times each ID appears.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

When do you need a dictionary instead of only a set?

Answer in your own words before continuing.

DE04-15 - Stack, queue, search and sort basics

What you will be able to do

Understand stack/queue behavior plus basic search/sort ideas well enough to recognize them in engineering code.

Why this matters

Job interviews and real systems use these concepts, but this module only needs practical intuition.

Picture it this way

A stack is a pile of plates: last plate placed is first removed (LIFO). A queue is a line at a counter: first person in is first served (FIFO).

Start from zero

- A Python list can demonstrate a simple stack with `append()` and `pop()`.
- For a real queue with efficient operations from both ends, `collections.deque` is usually a better fit than repeatedly removing index 0 from a list.
- **linear search** checks items one by one until a target is found.
- **sorted(values)** returns a sorted new list; `values.sort()` changes the list in place.
- Know the concepts; do not write production sorting algorithms from scratch when built-ins are correct and optimized.

Teacher demonstration

```
stack = []
stack.append("extract")
stack.append("transform")
stack.append("load")
last_step = stack.pop()
print(last_step)
print(stack)
```

Read the code like English

- load was appended last, so `pop()` removes load first.
- That is LIFO behavior.
- The remaining stack contains extract, transform.

Expected output / validation

output	result
popped step	load
remaining	['extract', 'transform']

Common beginner traps

- Calling every list a stack regardless of how it is used.
- Using `list.pop(0)` repeatedly for a large queue without knowing `deque` exists.
- Using `.sort()` when you needed to preserve the original list.

Now you do a similar task

Use a stack for three pipeline steps, explain LIFO, then write `linear_search(values,target)` returning index or -1. Also create `sorted_nums=sorted(nums)` and prove `nums` itself is unchanged.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

What is the practical difference between `sorted(nums)` and `nums.sort()`?

Answer in your own words before continuing.

DE04-16 - Integrated Python checkpoint and reproducible script habits

What you will be able to do

Combine Python fundamentals into a small rerunnable utility with separate read, validate, transform and output steps.

Why this matters

The goal of Python foundations is not isolated syntax. It is the ability to create a small script another person can run and understand.

Picture it this way

A reliable script is an assembly line with labeled stations instead of one person doing everything in one tangled motion.

Start from zero

- Separate responsibilities into functions: read input, validate rows, calculate/transform, write/print result.
- Use pathlib for file locations and keep raw input unchanged.
- Validate required columns/values before calculating.
- Use `__name__ == '__main__'` as the executable entry point.
- Print or log useful summary evidence such as input rows and output totals.
- Rerun the script from a clean state and confirm it produces the same result.

Teacher demonstration

```
from pathlib import Path
def main():
    input_path = Path(__file__).with_name("module_practical_orders.csv")
    # rows = read_rows(input_path)
    # validate_rows(rows)
    # result = completed_sales_by_region(rows)
    # print(result)
if __name__ == "__main__":
    main()
```

Read the code like English

- The skeleton intentionally leaves the practical functions for you.
- `main()` coordinates the steps but should not contain every detail.
- The supplied practical control totals are East 1000, North 1200, South 1250, West 2600.

Expected output / validation

region	Completed sales
East	1000
North	1200
South	1250
West	2600

Common beginner traps

- Hardcoding the expected totals instead of calculating them.
- Overwriting the raw CSV.
- Putting all logic into main() with no testable functions.
- Calling a script reproducible after running it only once.

Now you do a similar task

Complete DATA/module_practical_TODO.py only after lessons/practice. Run it twice, validate totals independently, and explain the role of each function.

How to prove it

- Run the code yourself.
- Check at least one expected output/type/row count.
- If your output differs, inspect the exact error/value before changing random code.

STOP - check your understanding

What makes a script reproducible besides 'it worked once on my laptop'?

Answer in your own words before continuing.